```
// The Task App may act only within CustCo-owned, installed workspac
es
permit(
  principal == ACME::Integration::"custco-tasks",
  action in [Action::"collab:task.read", Action::"collab:task.writ
e", Action::"collab:doc.meta.read"],
  resource in ACME::Workspace
)
when {
  resource.owner_tenant == principal.tenant &&
  resource.id in principal.allowed_workspaces
};

// Writes must carry a CustCo user attribution for audit/controls
permit(
  principal == ACME::Integration::"custco-tasks",
  action == Action::"collab:task.write",
  resource in ACME::Workspace
)
when { context.requested_by like /@custco.com$/ };
```

Because the target is `custco-sales` (an approved workspace) and the request includes `requested_by=jack@custco.com`, Cedar returns `allow`, and the gateway forwards the API call to create the task. If the app tries to write to a workspace outside its installation (such as `otherco-marketing`) or omits `requested_by`, Cedar would return `deny`, and the gateway would block the request.

This example demonstrates how Client Credentials work with Cedar to go beyond tenant-level scoping: the gateway enriches the integration principal with installation attributes, and policies enforce detailed constraints—such as specific workspaces, audit attribution, or exclusions like sensitive document types—without assigning the integration to any individual.

11.3 From microservices to third parties: One policy story

In this chapter, we followed ACME through two very different authorization challenges. Internally, microservices multiplied rapidly, and the company